High-Level Design (HLD)
RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop

SYSTEM OVERVIEW

The proposed system is a Retrieval-Augmented Generation (RAG) based Customer Support Assistant designed to answer user queries using a PDF knowledge base. The system retrieves relevant contextual information from document chunks stored as embeddings in ChromaDB and generates responses through an LLM processing layer. Queries with insufficient confidence or missing context are escalated to a Human-in-the-Loop (HITL) module for resolution. This architecture improves response accuracy and enables scalable automated customer support.

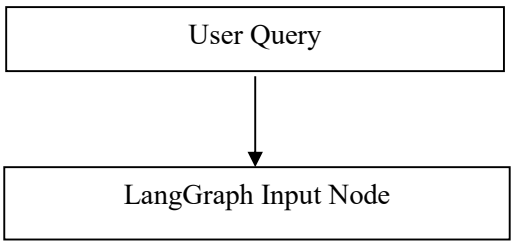
PROBLEM DEFINITION

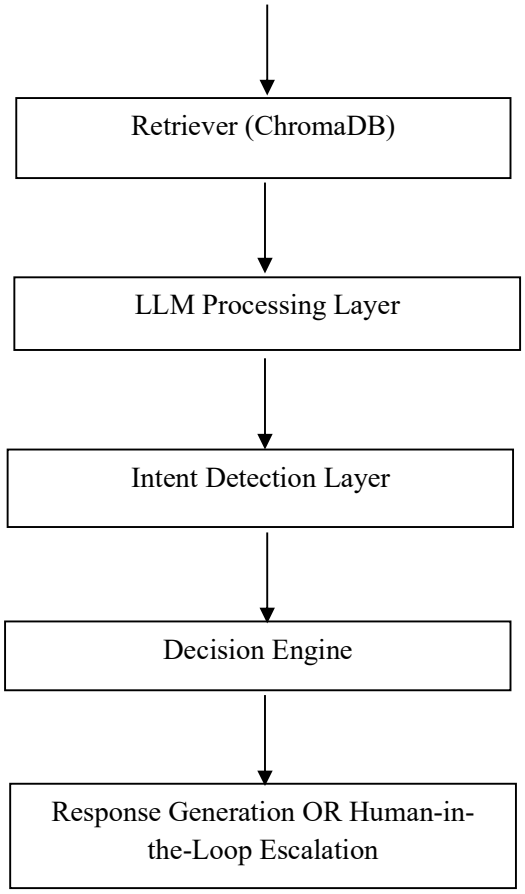
Traditional customer support chatbots often generate incorrect responses due to lack of contextual awareness. This system solves that limitation by retrieving relevant knowledge directly from a structured PDF knowledge base before generating responses. It also integrates workflow orchestration and escalation mechanisms to handle complex queries effectively.

SCOPE OF THE SYSTEM

The system processes PDF documents as a knowledge source and answers user queries using semantic retrieval techniques. It supports workflow orchestration through LangGraph and enables conditional routing for Human-in-the-Loop escalation. The system can be applied in customer support automation environments such as FAQ assistance, technical issue handling, and escalation-based service workflows.

SYSTEM ARCHITECTURE DIAGRAM





The architecture follows a Retrieval-Augmented Generation pipeline integrated with LangGraph workflow orchestration. User queries are processed through the input node and passed to the ChromaDB retriever to fetch relevant document chunks. The retrieved context is analyzed by the LLM processing layer to generate responses. An intent detection layer evaluates response confidence and routes complex queries to the Human-in-the-Loop escalation module when required.

COMPONENT DESCRIPTION

- **User Interface (CLI)**

The system provides a command-line interface that allows users to submit queries interactively. The interface forwards queries to the LangGraph workflow engine for processing and displays generated responses to the user.

- **Document Loader**

The Document Loader module loads the PDF knowledge base using PyPDFLoader and extracts textual content for further processing.

- **Chunking Strategy**

The Chunking Module divides extracted document text into smaller overlapping segments. A chunk size of 500 tokens with 50-token overlap is used to maintain contextual continuity between segments.

- **Embedding Model**

The Embedding Model converts document chunks into vector representations using Sentence Transformers, enabling semantic similarity search.

- **Vector Store (ChromaDB)**

ChromaDB is used to store embeddings efficiently and supports fast similarity-based retrieval operations.

- **Retriever**

The Retriever module performs similarity search on stored embeddings and returns the most relevant document chunks related to the user query.

- **LLM Processing Layer**

The LLM Processing Layer generates contextual responses using retrieved document chunks as supporting knowledge.

- **Graph Workflow Engine (LangGraph)**

LangGraph manages workflow orchestration between nodes including input handling, retrieval execution, decision routing, and response generation.

- **Routing Layer**

The Routing Layer evaluates response confidence and determines whether automatic response generation is sufficient or escalation is required.

- **Human-in-the-Loop (HITL) Module**

The HITL module handles complex queries by forwarding them to a human support agent who reviews the request and provides the final response.

DATA FLOW

The system operates in two main stages: document ingestion and query processing. During document ingestion, the PDF knowledge base is loaded, divided into chunks, converted into embeddings, and stored inside ChromaDB. During query processing, the user query is converted into an embedding and matched against stored vectors using similarity search. The retrieved chunks are passed to the LLM processing layer to generate responses. If the response confidence is low or context is insufficient, the query is escalated to a human support agent through the HITL module.

The query lifecycle begins when a user submits a query through the interface, after which the system retrieves relevant document chunks, evaluates response confidence, and either generates an automated response or escalates the request through the Human-in-the-Loop module.

TECHNOLOGY CHOICES

ChromaDB is selected as the vector database due to its lightweight architecture and efficient similarity search capabilities. LangGraph is used for workflow orchestration between processing nodes. Sentence Transformers are used for generating semantic embeddings. Python is chosen as the implementation language due to its strong ecosystem support for artificial intelligence and machine learning applications.

SCALABILITY CONSIDERATIONS

The system supports scalability through modular architecture and efficient vector retrieval mechanisms. It can be extended to support multiple PDF documents and larger datasets. Future enhancements may include migration to cloud-based vector databases such as Pinecone, asynchronous query handling, and distributed deployment for handling high query loads.